DB 구조

테이블은 13개. 핵심은 은행과 카드사를 fin_companies 한 테이블에 넣은 것과, 우대조건 원문을 product_pref_conditions에 항목 단위로 쪼개 넣은 것이다. 이 두 가지 덕분에 "이 적금은 우리 카드 실적이 있으면 금리를 더 준다"를 JOIN으로 계산할 수 있다.

1. 요구사항 구체화

수업 방식대로 문장마다 명사는 테이블·열 후보, 동사는 관계로 표시했다.

#	요구사항	테이블 (명사)	관계 (동사)
1	금융사는 은행, 저축은행, 카드사로 나뉘고 같은 금융그룹끼리 묶인다	fin_companies	—
2	금융사는 예적금 상품을 여러 개 판다	deposit_products	금융사 1 : 상품 N
3	상품 하나도 저축기간과 적립방식(정액/자유)마다 금리가 다르다	rate_options	상품 1 : 옵션 N
4	상품에는 우대조건이 여러 개 있고, 조건마다 가산금리가 붙는다	product_pref_conditions	상품 1 : 조건 N
5	우대조건은 정해진 유형 중 하나다 (급여이체, 카드실적, 자동이체, 첫 거래, 비대면 가입, 마케팅 동의)	pref_condition_types	유형 1 : 조건 N
6	카드실적 조건은 어느 카드사 카드를 월 얼마 이상 쓰는지 정해져 있다	product_pref_conditions의 card_company_id, required_amount	카드사 0..1 : 조건 N
7	카드사는 카드를 여러 장 발급한다	cards	금융사 1 : 카드 N
8	카드는 카테고리별 혜택을 가진다	card_benefits, benefit_categories	카드 M : N 카테고리
9	회원은 카테고리별 월 지출을 입력한다	user_spending	회원 M : N 카테고리
10	회원은 어느 금융사에서 어떤 조건을 충족하는지 체크한다 (예: 신한은행 급여이체)	user_conditions	회원 1 : N, 유형 1 : N
11	회원은 카드 + 적금 조합을 저장한다	saved_combos	회원 1 : 조합 N
12	금리는 매일 수집해서 이력으로 남긴다	rate_history	옵션 1 : 이력 N

리뷰, 이벤트, 조회 기록은 card_compare_db.sql 구조를 그대로 가져가서 여기서는 뺐다.

2. 테이블과 PK 설정

기본은 의미 없는 일련번호(인조식별자)를 PK로 두고, 업무상 유일해야 하는 값에 UNIQUE를 건다. 복합키는 user_spending 하나만 쓴다.

테이블	PK	UNIQUE	이렇게 정한 이유
fin_companies	company_id	fin_co_no	API 금융사 코드는 카드사에 없어서 PK로 못 쓴다. 있을 때만 중복 금지
deposit_products	product_id	(company_id, fin_prdt_cd)	상품코드는 금융사끼리 겹칠 수 있어서 금융사와 묶어야 유일하다
rate_options	option_id	(product_id, save_term, rsrv_type, intr_rate_type)	같은 12개월이라도 정액/자유, 단리/복리마다 금리가 다르다
pref_condition_types	type_id	name	코드값 성격의 작은 테이블
product_pref_conditions	pc_id	—	한 상품에 같은 유형 조건이 두 번 나올 수 있다 (자동이체 2건 등). 복합키로 만들면 막혀 버린다
cards	card_id	(company_id, card_name)	카드사 안에서 이름 중복 금지
benefit_categories	category_id	name	
card_benefits	benefit_id	—	한 카드에 같은 카테고리 혜택이 여러 줄일 수 있다 (스타벅스 30%, 그 외 카페 10%)
users	user_id	email	
user_spending	(user_id, category_id)	—	회원 × 카테고리 = 딱 한 줄. 번호를 따로 붙일 이유가 없다
user_conditions	uc_id	(user_id, type_id, company_id)	company_id가 비어 있을 수 있는 조건(마케팅 동의 등)이 있어 복합 PK가 안 된다. PK는 NULL 불가
saved_combos	combo_id	(user_id, card_id, option_id)	같은 조합 중복 저장 금지
rate_history	history_id	(option_id, collected_on)	하루 한 번 수집

3. 핵심 테이블 구상

이 서비스에서 새로 고민한 테이블은 두 개다. 나머지는 카드 DB와 API 구조를 거의 그대로 따른다.

product_pref_conditions (상품별 우대조건) — 원문 한 문장을 조건 하나당 한 줄로 쪼갠다.

컬럼	자료형	제약	설명
pc_id	INT	PK, AUTO_INCREMENT	
product_id	INT	FK, NOT NULL	어느 상품의 조건인지
type_id	INT	FK, NOT NULL	급여이체, 카드실적 …
bonus_rate	DECIMAL(4,2)	NOT NULL, CHECK > 0	가산금리 (%p)
card_company_id	INT	FK, NULL 허용	카드실적 조건일 때만. 어느 카드사 카드인지
required_amount	INT	NULL 허용	기준 금액 (원). 카드 월 30만 원, 급여 월 50만 원 등
description	VARCHAR(255)		쪼개기 전 원문 조각 (검증용)

예를 들어 원문이 "급여이체 시 연 0.3%p, 신한카드 월 30만 원 이상 사용 시 연 0.5%p"라면 두 줄이 된다.

pc_id	product_id	type_id	bonus_rate	card_company_id	required_amount
1	101	급여이체	0.30	NULL	NULL
2	101	카드실적	0.50	신한카드	300000

user_conditions (회원이 충족하는 조건) — 회원이 체크한 상황을 저장하고, 위 테이블과 type_id, company_id로 맞춰 본다.

컬럼	자료형	제약	설명
uc_id	INT	PK, AUTO_INCREMENT	
user_id	INT	FK, NOT NULL	
type_id	INT	FK, NOT NULL	어떤 조건인지
company_id	INT	FK, NULL 허용	어느 금융사에서 충족하는지 (급여이체 은행 등)

카드실적 조건은 회원이 체크하지 않는다. 회원이 고른 카드의 카드사가 card_company_id와 같고, 총지출이 required_amount 이상이면 충족으로 계산한다. 조합마다 결과가 달라지는 이유가 여기 있다.

ERD


deposit_products
PK product_id
FK company_id
fin_prdt_cd (API 상품코드)
product_name
product_type (예금/적금)
pref_text (우대조건 원문)
fin_companies
PK company_id
UK fin_co_no (API 코드)
name
company_type (은행/카드사)
group_name (금융그룹)
cards
PK card_id
FK company_id
card_name
card_type (신용/체크)
min_spending (전월실적)
카드실적 조건
rate_options
PK option_id
FK product_id
save_term (개월)
rsrv_type (정액/자유)
base_rate (기본금리)
max_rate (최고금리)
product_pref_conditions
PK pc_id
FK product_id
FK type_id
FK card_company_id
bonus_rate (가산금리)
required_amount (기준액)
card_benefits
PK benefit_id
FK card_id
FK category_id
rate (혜택률)
monthly_limit (월 한도)
pref_condition_types
PK type_id
UK name (급여이체, 카드실적 …)
users
PK user_id
UK email
nickname
birth_year
benefit_categories
PK category_id
UK name (교통, 커피, 쇼핑 …)
user_conditions
PK uc_id
FK user_id
FK type_id
FK company_id (어느 금융사)
표기 (IE 까마귀발)
막대 하나 · 세 갈래 여럿
동그라미 = 없어도 됨 (NULL)
FK는 항상 여럿(N) 쪽에 둔다
user_spending
PK, FK user_id
PK, FK category_id
monthly_amount (월 지출)